Cordis, Spatiotemporal Composability and Flow-Based Programming

Research findings and conditional agent-design lessons

Research-phase — not battle-tested. Source snapshot: 29 October 2026; synthesis: 30 October 2026.

Reader contract	Value
Audience	Technically literate maintainers and agents; familiarity with modules, dependencies and asynchronous execution is assumed.
Purpose	Understand two distinct composition models and evaluate optional questions for design and implementation.
Scope	Primary-source findings, limitations, conditional guidance and one pilot; no framework adoption, production promotion or performance claim.

Read first: lifecycle cleanup is not arbitrary rollback; bounded buffers do not guarantee progress; context controls do not sandbox hostile code. No local accepted use or measured agent improvement is claimed.

Contents

1 Two complementary questions 2
1.1 What “spatiotemporal” means here 2
1.2 Cleanup ordering is narrower than “LIFO everywhere” 2
1.3 Canonical flow-based programming 2
1.4 Complementarity, not an integration theorem 3
2 Where the guarantees stop 4
2.1 Reversible acquisition versus irreversible emission 4
2.2 Confluence is not reproducible agent output 4
2.3 Dependency controls are not a sandbox 4
2.4 Names and reloads do not solve state compatibility 4
2.5 Bounded buffers can still deadlock 4
3 Conditional lessons for design and materialization 5
3.1 Questions to carry through the work 5
3.2 Dataflow questions remain separate 6
3.3 One bounded pilot before any promotion 6
3.4 Disconfirming examples 6
4 Evidence, versions and open questions 7
4.1 Source priority and snapshot 7
4.2 Secondary-source corrections 7
4.3 What has and has not been established 7
Sources 9

1 Two complementary questions

Cordis is a TypeScript plugin meta-framework with roots in Koishi. Its paper presents a calculus and an implementation for composing components through tracked effects and reactive environment requirements; it does not make every program reversible. [1], [2]

View	Conceptual progression
Lifecycle / dependency	Provider becomes available ⟶ consumer can activate ⟶ dependent stops before provider disappears.
Dataflow	Producer output port ⟶ buffered connection ⟶ consumer input port.

Table 1: Two explanatory views, not a proposed runtime architecture. The first concerns component availability and cleanup; the second concerns packet movement and flow control.

1.1 What "spatiotemporal" means here

Spatial composition concerns an environment's available dependencies: a component declares what it needs, activates when those requirements are satisfied, and deactivates before they cease to be satisfied. It is not geographic placement, deployment sharding or distributed scheduling. [2]

Temporal composition concerns a component's lifetime and the cleanup of its tracked effects. Removing a component should remove its mediated registrations and acquired resources within the model's boundary, rather than leave a listener, timer or handle behind; it is not a promise to reverse arbitrary external history. [2]

An **effect** describes what a computation does to its environment; a **coeffect** describes what it requires from that environment. Cordis uses a context to mediate both, connecting dependency resolution with effect ownership and lifecycle. [2]

1.2 Cleanup ordering is narrower than "LIFO everywhere"

The inspected `packages/core/src/fiber.ts` collects disposers inside one `ctx.effect` execution and runs that registration's collected cleanup in reverse order, sequentially. At fiber unload, however, separate top-level effect disposers are started through `Promise.all`; paper §5.1.3, p. 63, explicitly describes concurrent effect disposal. [2], [3]

These are different ordering scopes. A consumer must not rely on an unstated cross-effect teardown sequence merely because one registration has reverse-order cleanup; the atomic inverse itself remains an author obligation (§5.1.1, p. 59). [2]

1.3 Canonical flow-based programming

Morrison's FBP describes independent asynchronous black-box process instances connected through named ports. Information packets travel through externally defined connections, with ownership/lifetime rules and bounded buffering; subnetworks support hierarchical composition. [4]

The network definition describes connectivity rather than hiding it inside each component's code. Backpressure constrains a producer when its receiving connection cannot accept more data; concrete implementations can also expose dropping policies, so their actual contracts must be inspected rather than inferred from a diagram. [4]

An arbitrary task DAG is not automatically FBP: it may describe finite task dependencies without asynchronous packet processes or bounded channels. An event bus, reactive stream or graph picture likewise does not establish the whole model; Morrison's deadlock discussion includes I/O and external coordination, so FBP should not be read as universally prohibiting side effects. [4], [5]

1.4 Complementarity, not an integration theorem

Lifecycle/dependency composition asks when a component may exist and what must be cleaned up when it leaves. FBP asks how data moves among active components and what happens when consumers are slower than producers. [2], [4]

Calling these a “control plane” and “data plane” can help explain the distinction, but that is an analytical analogy, not a proof that Cordis plus an FBP runtime forms a safe or useful combined system. Neither source establishes the combined agent architecture or its benefit to this setup.

2 Where the guarantees stop

2.1 Reversible acquisition versus irreversible emission

Paper §6.1, pp. 70–71, puts a location inside the reversible boundary only when the system can both modify it exclusively and restore its prior state. The boundary is per location, not per storage medium: a private scratch file and a shared file can have different status. [2]

Opening a descriptor can be paired with closing it. Sending bytes, publishing an email or modifying shared external state does not become retractable merely because the component subsequently unloads; these emissions lie outside the paper's tracked reversion boundary. [2]

The paper discusses withholding output until it can persist and application-defined compensation, such as refunding a charge. Compensation restores a chosen business equivalence, not necessarily the original state; §6.1 explicitly says the metatheory must be re-established for the coarser equivalence rather than inherited automatically. [2]

2.2 Confluence is not reproducible agent output

The lifecycle-confluence result (§4.3.5, p. 51) concerns lifecycle transitions for the same orchestration inputs under the formal model's assumptions. Different orchestration inputs need not have equivalent outcomes; the result does not prove deterministic LLM responses, exactly-once external effects or distributed consistency. [2]

Termination arguments likewise have conditions, including finite fibers and the stated dependency/iteration assumptions. Dependency cycles can leave components inactive (§6.5); the theorem is not a license for unbounded self-instantiation or safe arbitrary self-modification. [2]

2.3 Dependency controls are not a sandbox

Paper §6.3, pp. 72–73, distinguishes restricting declared dependency access from executing malicious code safely. Contexts, interception and injection address the former; hostile code in the host runtime can bypass language-level mediation, so the latter needs an external execution boundary. [2]

Neither FBP process ownership nor an independently named component necessarily means a separate operating-system process or security boundary. Packet ownership is a communication contract, not evidence of hostile-code isolation. [4]

2.4 Names and reloads do not solve state compatibility

Paper §6.6, pp. 75–76, explains that a nominal service key does not establish structural or behavioral compatibility. Peer dependencies and semantic-version conventions help coordinate versions but do not constitute a general dynamic compatibility proof. [2]

Reloading by revert-and-reapply does not preserve arbitrary component-local memory. State that must survive needs a longer-lived owner or an explicitly established migration/recovery contract; the related-work discussion (§7.3) distinguishes this from general dynamic state migration. [2]

2.5 Bounded buffers can still deadlock

Morrison gives a useful counterexample: a consumer wants a count before it starts receiving records, but the count is produced only after all records are sent. When the bounded record channel fills, the producer cannot finish and emit the count, while the consumer refuses the records until it receives that count. [5]

Feedback loops and split/recombine topologies can therefore require careful receive ordering and packet-count reasoning. The buffer bound constrains storage and creates flow control; it does not prove progress, absence of deadlock or appropriate overload behavior for a particular runtime. [5]

3 Conditional lessons for design and materialization

Status: these questions are optional research guidance, not accepted production instructions or a new approval gate. Apply them when component/plugin lifetimes, changing dependencies, streaming channels or rollback claims are actually involved; skip the extra machinery for straightforward linear code.

Existing implementation guidance remains the owner of ordinary execution-flow practice: inspect component/channel ownership, buffer bounds, backpressure, ordering and termination inside an approved stack. The research does not authorize replacing that stack, adopting Cordis/NoFlo, adding a second orchestrator or implementing a custom runtime.

3.1 Questions to carry through the work

Check	Apply when	Evidence to record	Stop signal
Separate the axes	A design invokes spatiotemporal composition.	Which claim concerns dependencies/lifetimes, and which concerns packet flow.	The term means scheduling or geography without qualification.
Declare requirements	Effects need context or services.	Dependencies and required capabilities at the relevant effect boundary.	A required dependency is hidden in global state.
Explain lifecycle order	Providers can appear, disappear or reload.	Load dependencies before consumers; stop dependents before providers.	A dependent can outlive an unavailable provider without a defined contract.
Name cleanup scope	An implementation relies on teardown ordering.	Actual selected API/version; distinguish within-effect order from cross-effect concurrency.	Assumed cross-effect LIFO without source evidence.
Bound the inverse claim	A feature promises rollback or cleanup.	Exact inverse, compensation, or explicitly unrestorable effect; equivalence used.	An external emission is described as automatically undone.
Locate durable state	A component owns state or reloads.	Exclusive/restorable versus shared/external state; what reload loses.	Required state has no surviving owner or recovery contract.
Separate security claims	A component is untrusted.	External execution boundary and its actual protection, separately from dependency controls.	Contexts or packet ownership are called a sandbox.
Check compatibility	A service provider is selected or replaced.	Required structural/ behavioral contract, not merely matching service names.	Name/version equality is treated as proof of behavior.

These checks are derived from the inspected paper and implementation, not measured guidance effectiveness. They do not require a universal form or a separate report: preserve relevant observations in the work's existing design/implementation handoff. [2], [3]

3.2 Dataflow questions remain separate

For a concurrent streaming workload, the existing execution-flow lens already asks about component boundaries, ownership, buffer bounds, backpressure, ordering and termination. Canonical FBP provides vocabulary for that inspection; a bounded queue or a DAG diagram alone supplies none of the missing behavioral evidence. [4]

For example, the count-before-records case in the preceding chapter disconfirms "bounded means safe." Removing a lifecycle-managed consumer also says nothing, by itself, about pending packets or in-flight work: the communication and shutdown contracts still have to be known, rather than inferred from the two models' names.

3.3 One bounded pilot before any promotion

Comparison: on one authorized real design/materialization task, apply the existing guidance and these optional questions to the same source inputs and acceptance contract. Include a relevant failure case, such as irreversible emission or an unavailable component; do not execute unsafe effects merely to test the questions.

Observations: record any concrete defect the baseline missed, any false alarm or unnecessary machinery the questions added, and whether the evidence was obtainable. A source walkthrough can check comprehension, but must remain labeled a walkthrough rather than an executed lifecycle test or proof of improved agent behavior.

Decision boundary: retain research status until independent review and an accepted use support promotion under the setup's existing promotion process. There is no accepted local use yet; absence of an observed defect or successful document compilation does not establish effectiveness.

3.4 Disconfirming examples

- **No rollback:** unloading a plugin after sending an email does not unsend it.
- **No global LIFO:** two separate fiber effects may begin cleanup concurrently.
- **No progress proof:** a count-before-records protocol can block despite bounded channels.
- **No behavioral proof:** equal service names do not establish compatible behavior.
- **No output determinism:** equivalent lifecycle configurations need not generate the same model response.

These examples are desk checks of the interpretation, not results from running a Cordis/FBP agent system.

4 Evidence, versions and open questions

4.1 Source priority and snapshot

The research inspected the paper, upstream Cordis materials, DSH's documentation/vendor manifest and Morrison's original FBP explanations. The AgentAtlas article is a secondary introduction, not the authority for theorem scope, implementation behavior or chronology. [1], [2], [4], [6], [7], [8]

Source	Observed scope and limitation
Paper	arXiv 2608.25512 v1, submitted 26 August 2026, 92 pages. Formal claims are scoped to the model and assumptions; proofs were not independently mechanized or re-proved in this research.
Upstream Cordis	Inspected main tree reported 4.0.0-rc.12 and unstable API status. This is a tree snapshot, not verification of a release tag or immutable source pin.
DeepSeek Harness	Vendor manifest reports `@deepseek-ai/cordis` 4.0.0-rc.7, upstream commit 56b3d4f725681cf4556c1a8695a709cc3b6eed74 and 24 local modifications. A modified vendored tree is not interchangeable with upstream main.
Koishi case study	Paper §5.3 reports more than 4000 community plugins over four years. Its footnote says Koishi uses Cordis V3, whereas the paper concerns V4; adoption is observational evidence, not a controlled quality or safety comparison.
FBP	Morrison's conceptual account and deadlock discussion support the model and counterexample, not claims about every current runtime's buffering policy.

The report preserves these observations as a dated snapshot, not a promise about today's moving branches. Before adopting an API or comparing behavior, inspect and pin the exact intended artifact; no Cordis or FBP runtime was installed or run for this research.

4.2 Secondary-source corrections

The AgentAtlas post is dated 17 August 2026 and places the preprint earlier than arXiv's recorded v1 submission. Use arXiv's 26 August metadata for the submission date; the post's claimed 13 August DSH release is a secondary attribution, not independently established release evidence. Repository creation is not proof of release. [2], [8]

An initial delegated interpretation alleged that the article's nested `ctx.effect` cleanup example leaked. Original-source proofreading rejected that allegation: the outer execution thunk is called immediately and its returned cleanup is collected for teardown; the example is not evidence of that leak. [3], [8]

This research does not establish the current buffering behavior of NoFlo or Node-RED. It makes no current unbounded-queue claim for either, and does not generalize all reactive programming to one scheduling model.

4.3 What has and has not been established

Established at source level: the distinction between lifecycle/dependency composition and packet-flow composition; scoped cleanup ordering; the paper's reversibility, compatibility and security limits; and a documented bounded-buffer deadlock counterexample. Independent proofreading corrected delegated overclaims rather than treating an agent response as evidence by itself.

Not established: local Cordis/DSH behavior, performance, security or recovery; correctness of a combined Cordis–FBP agent architecture; runtime equivalence across versions; actual agent use of the

optional links; or an improvement in modularity, flexibility or output quality. Those are future hypotheses, not conclusions of this document.

Open questions: which concrete workload needs dynamic lifecycle composition; whether a runtime is necessary at all; how its dataflow shutdown and durable state contracts interact; and whether the optional questions detect useful defects without adding noise. These require task-specific evidence and existing approval boundaries, not blanket adoption.

Sources

[1] Cordis contributors, “Cordis repository and framework documentation.” Accessed: Oct. 29, 2026. [Online]. Available: https://github.com/cordiverse/cordis

[2] Y. Shi, W. Zhang, and T. Cui, “A Programming Paradigm for Spatiotemporal Composability.” Accessed: Oct. 29, 2026. [Online]. Available: https://arxiv.org/abs/2608.25512v1

[3] Cordis contributors, “Core fiber implementation.” Accessed: Oct. 29, 2026. [Online]. Available: https://github.com/cordiverse/cordis/blob/main/packages/core/src/fiber.ts

[4] J. P. Morrison, “Flow-Based Programming: Concepts.” Accessed: Oct. 29, 2026. [Online]. Available: https://jpaulm.github.io/fbp/concepts.html

[5] J. P. Morrison, “Flow-Based Programming: Deadlocks.” Accessed: Oct. 29, 2026. [Online]. Available: https://www.jpaulmorrison.com/fbp/deadlock.shtml

[6] DeepSeek Harness contributors, “Vendored packages manifest.” Accessed: Oct. 29, 2026. [Online]. Available: https://github.com/deepseek-ai/deepseek-harness/blob/master/vendor/README.md

[7] DeepSeek Harness contributors, “Cordis primer.” Accessed: Oct. 29, 2026. [Online]. Available: https://github.com/deepseek-ai/deepseek-harness/blob/master/docs/cordis-primer.md

[8] Lucy, “Cordis Explained: How DeepSeek Harness's Plugin Framework Works.” Accessed: Oct. 29, 2026. [Online]. Available: https://agentatlas.org/blog/cordis-explained-how-deepseek-harness-plugin-framework-works/